IMPERIAL

Seq2Seq

Shamsuddeen Muhammad
Google DeepMind Academic Fellow,
Imperial College London
https://shmuhammadd.github.io/

Reference

These slides are based on the course material by Daniel Jurafsky :

Chapter 08: https://web.stanford.edu/~jurafsky/slp3/

Bi-directional RNN

Traditional RNNs process sequences in a single direction, typically from past to future.

This is fine for **language modeling**, where predicting the next word only needs left context.

While this approach captures historical dependencies, it may overlook future context, which can be crucial for tasks where understanding both past and future information is essential.

For instance, in speech recognition, accurately interpreting a word can depend on both its preceding and succeeding words.

But for many NLP tasks (e.g., NER, POS tagging, translation, QA), future context is also important.

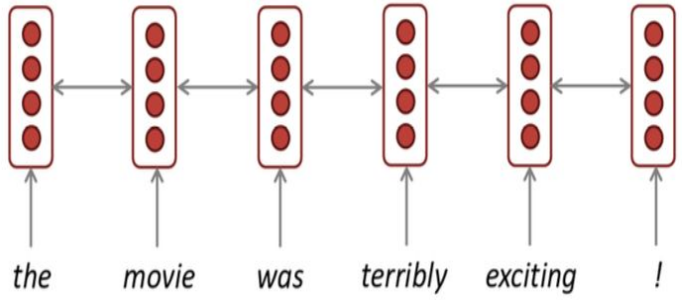
Bi-directional RNN

Bidirectional RNNs extend standard RNNs by processing a sequence in **two directions**:

(1) **Forward** (left → right) and
(2) **Backward** (right → left).

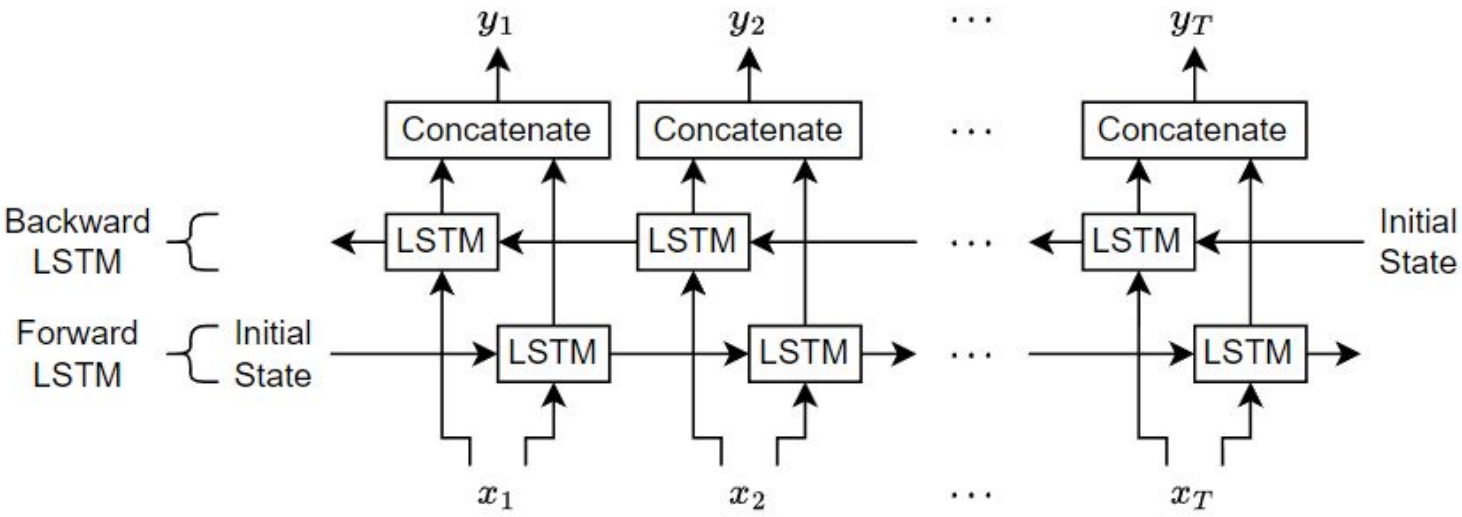
At each time step, the model combines information from both directions, producing a context-rich representation that incorporates **past and future** dependencies.

This contrasts with a unidirectional RNN, which only has access to past tokens.



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states

Bi-directional RNN



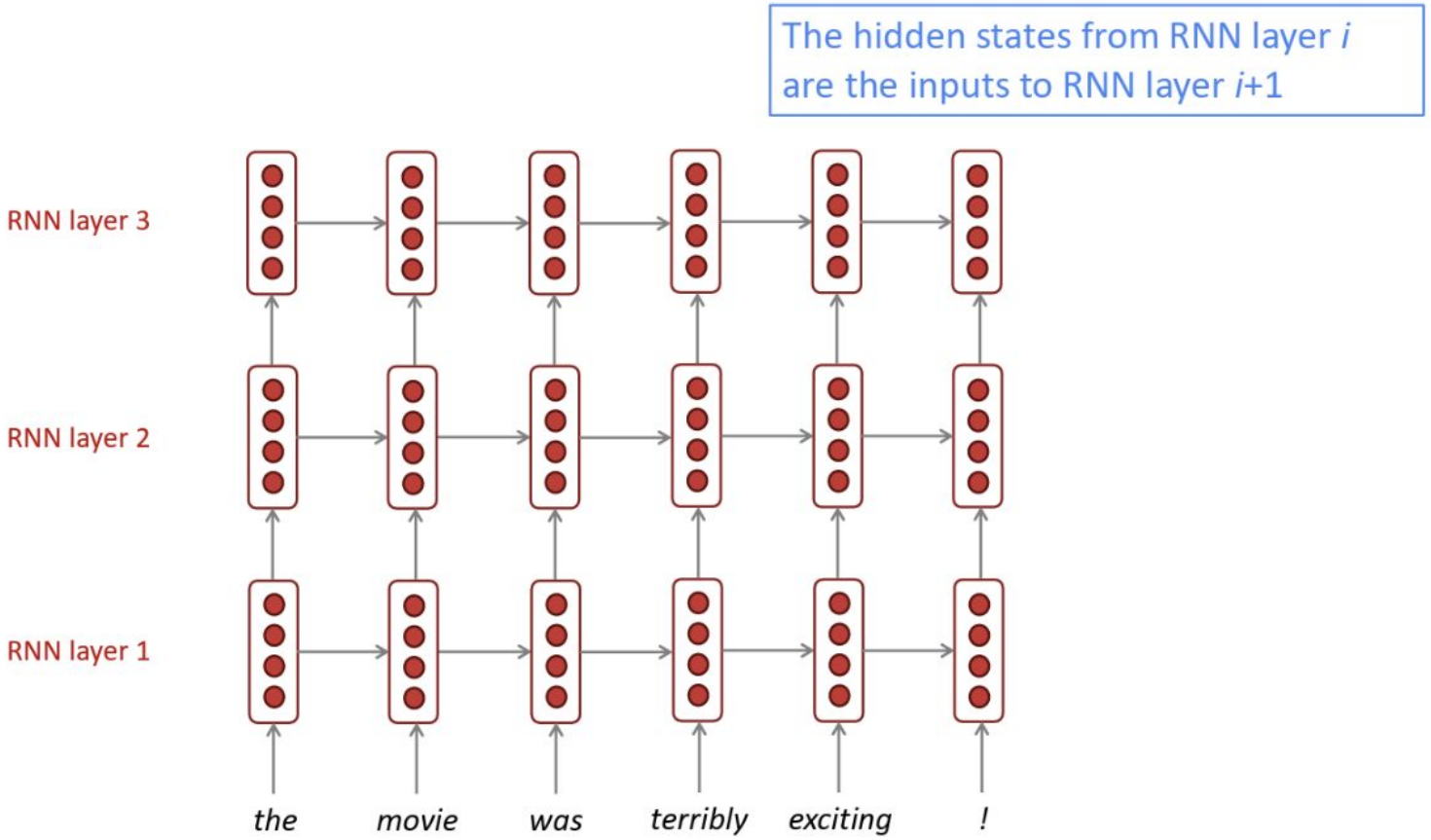
BiLSTM: Bidirectional Long Short-Term Memory

BiGRU: Bidirectional Gated Recurrent Unit

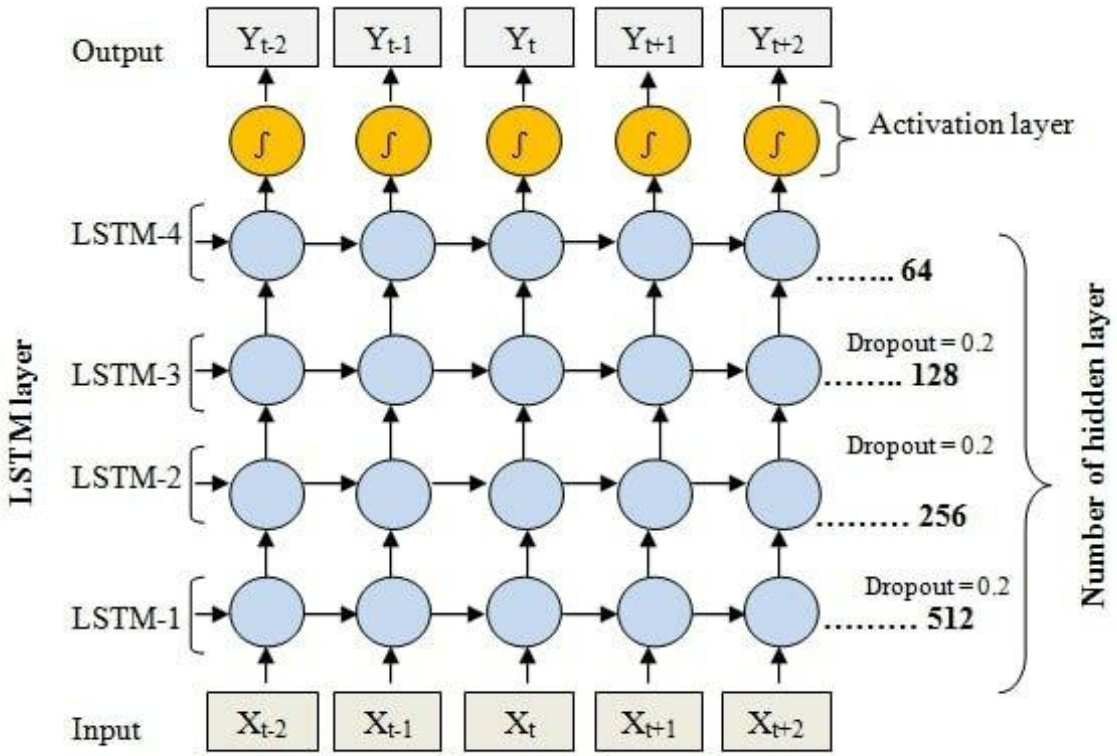
Capture Long-range dependencies

- RNNs are already "deep" on one dimension (they unroll over many timesteps)
- We can also make them "deep" in another dimension by applying multiple RNNs – this is a multi-layer RNN.
- This allows the network to compute more complex representations
  - The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features.
- Multi-layer RNNs are also called *stacked RNNs*.

Capture Long-range dependencies



Capture Long-range dependencies



Capture Long-range dependencies

Multi-layer or stacked RNNs allow a network to compute more complex representations – they work better than just have one layer of high-dimensional encodings!

- The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features.

High-performing RNNs are usually multi-layer (but aren't as deep as convolutional or feed-forward networks)

Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.

LSTMs: real-world success in

In 2013–2015, LSTMs started achieving state-of-the-art results

- Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
- LSTMs became the dominant approach for most NLP tasks

Now (2019–2024), Transformers have become dominant for all tasks

- For example, in **WMT** (a Machine Translation conference + competition):
  - In WMT 2014, there were 0 neural machine translation systems (!)
  - In WMT 2016, the summary report contains "RNN" 44 times (and these systems won)
  - In WMT 2019: "RNN" 7 times, "Transformer" 105 times

LSTM

LSTMs mitigate vanishing gradients via gating (forget/input/output).

They improve learning of long-range dependencies vs. vanilla RNNs.

Yet, many tasks require mapping one sequence to another (not just labeling a single sequence).

LSTM

- Read the input sequence and use the final hidden state for prediction.

- Difficulty: **handling variable-length outputs.**

Sequence-to-Sequence Problems

Examples: translation, summarization, dialogue generation, speech-to-text.

What We Learned from LSTMs

LSTMs improve sequence modeling by remembering long-term dependencies.

LSTM allows models to:

- Process sequences token by token
- Maintain a memory cell to store important information
- Capture long-term dependencies better than standard RNNs

LSTM: Applications

Applications include:

- Sentiment classification
- Language modeling
- Speech recognition

However, these tasks typically produce a single output.

Limitation of Standard LSTM Models

Many real-world NLP tasks require sequence-to-sequence mapping, not single outputs.

Examples: translation, summarization, dialogue generation, speech-to-text.

Limitation of Standard LSTM Models

Many real-world tasks map a variable-length input sequence to a variable-length output sequence.

English sentence	→	French sentence	Translation
Question	→	Answer	QA
Document	→	Summary	Summarisation
Audio waveform	→	Text transcript	ASR

The student presented the research proposal

A student gives research presentation

How can a model generate an entire sequence as output?

Standard LSTM Is Not Enough

Input sequence → LSTM → Single output

Input sequence → Model → Output sequence

We want to have:

Input sequence → Model → Output sequence

Key Idea: Two LSTMs Instead of One

To solve this problem, researchers proposed the Encoder–Decoder architecture (Sutskever et al., 2014).

Encoder–Decoder uses one LSTM to understand the input sequence and another LSTM to generate the output sequence.

Encoder-Decoder

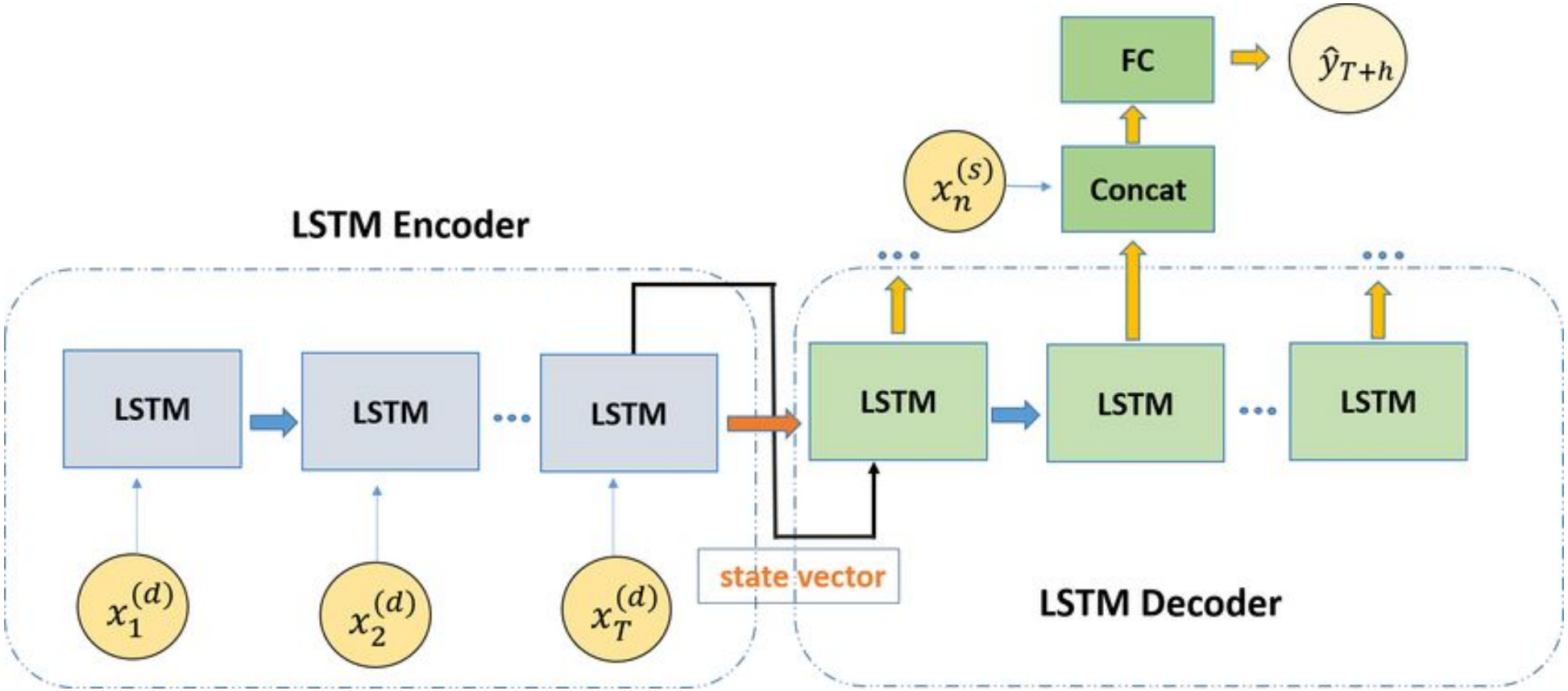
The idea:

1. Encoder LSTM
   - Reads the entire input sequence
   - Compresses it into a context vector

2. Decoder LSTM
   - Uses that vector to generate the output sequence

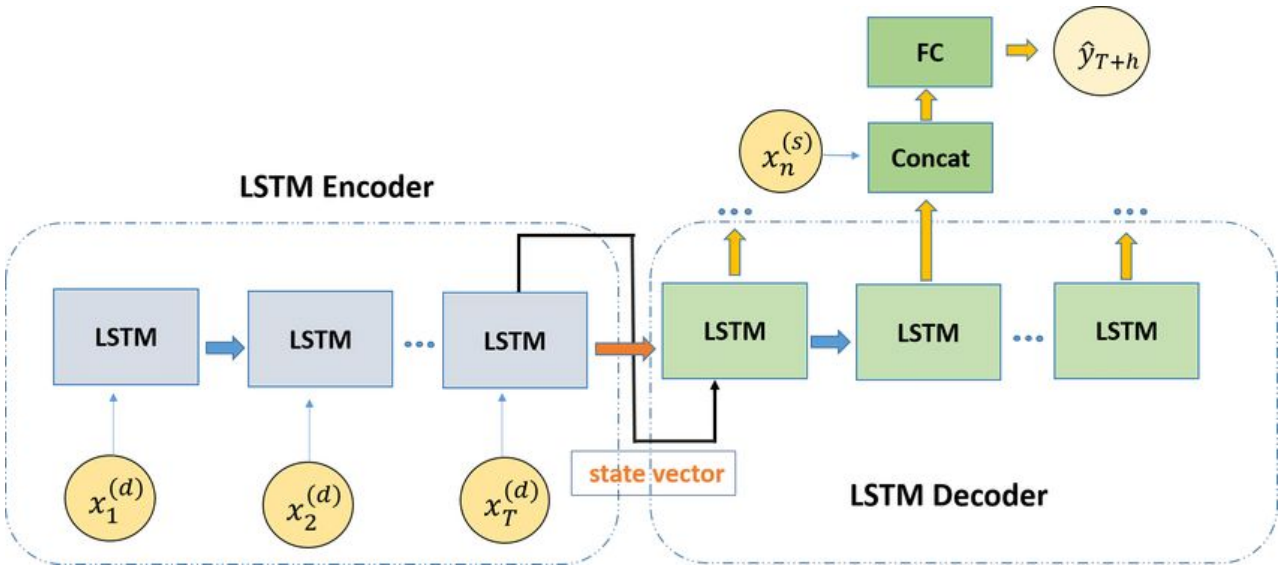
Conceptually: Input sequence → Encoder → **Context vector** → Decoder → Output sequence

Encoder-Decoder

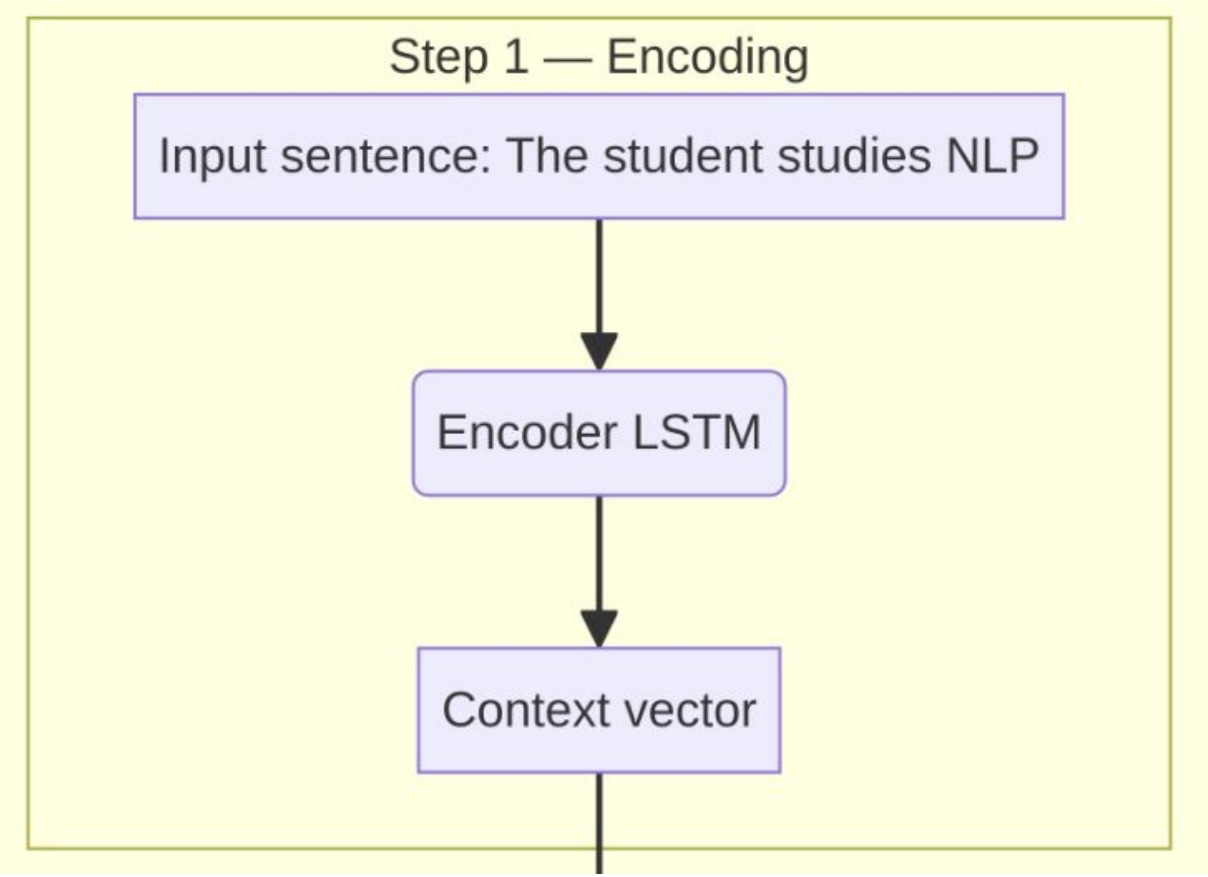


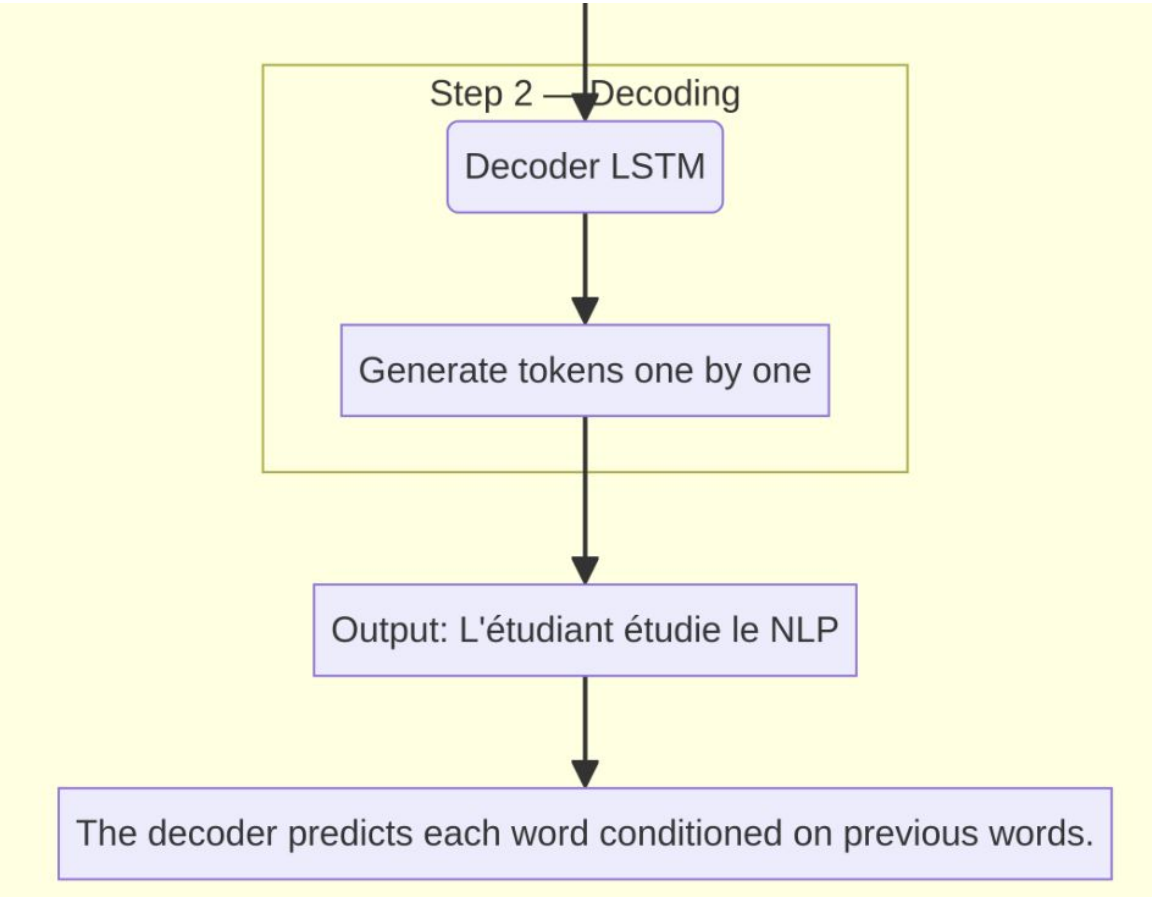
The Bridge Between LSTM and Encoder–Decoder

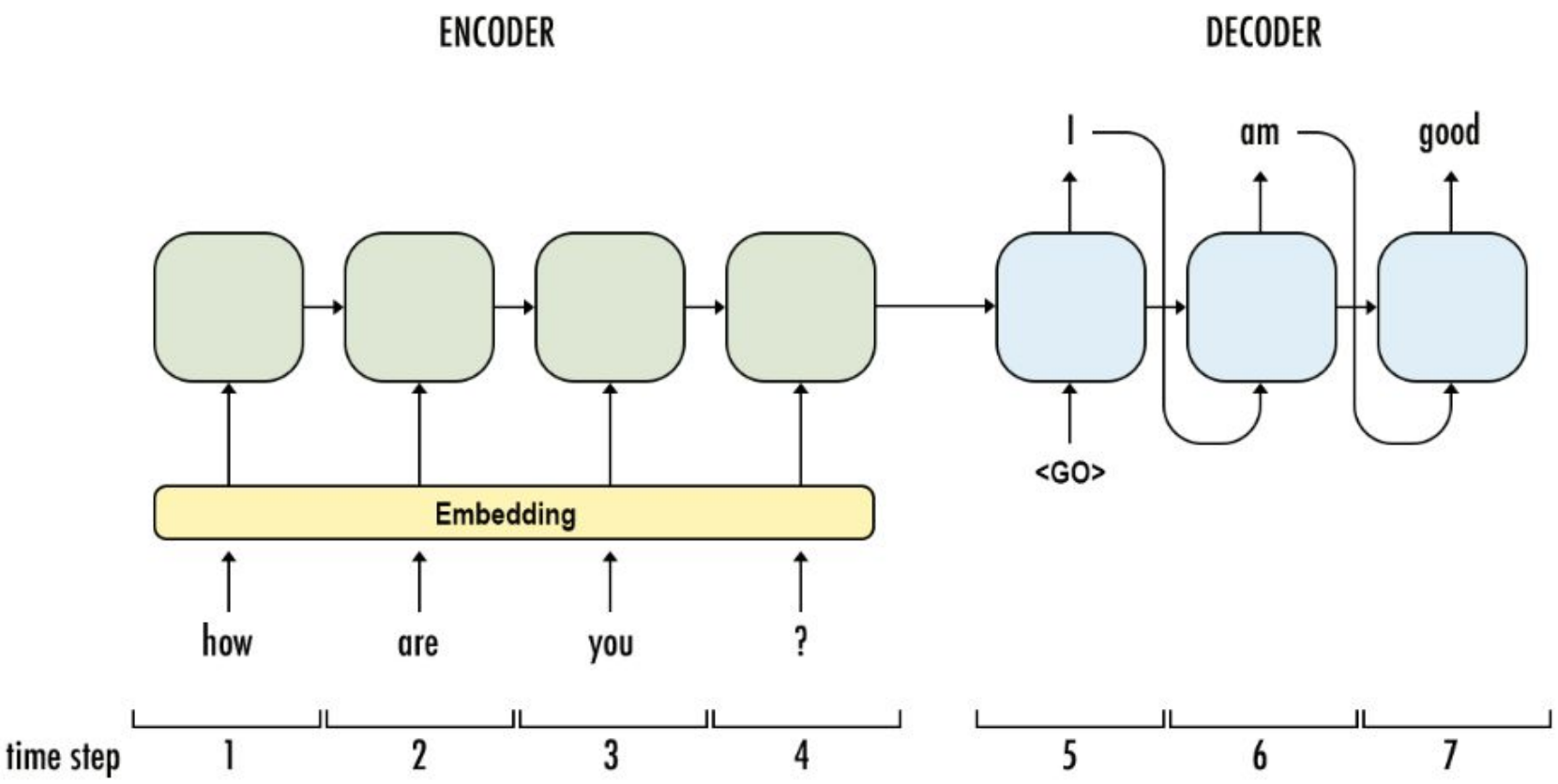
- The bridge is the **hidden state** produced by the LSTM.
- The final hidden state: This vector represents **the meaning of the whole sequence**.
- This hidden state becomes the **initial state of the decoder.**
- This vector is often called the **context vector** or **state vector**



Encoder–Decoder uses one LSTM to understand the input sequence and another LSTM to generate the output sequence.


Step 1 — Encoding
Input sentence: The student studies NLP
Encoder LSTM
Context vector


Step 2 — Decoding
Decoder LSTM
Generate tokens one by one
Output: L'étudiant étudie le NLP
The decoder predicts each word conditioned on previous words.


ENCODER
DECODER
I
am
good
<GO>
Embedding
how
are
you
?
time step
1
2
3
4
5
6
7

Encoder-Decorder

Solves the problem of **Sequence-to-Sequence (seq2seq)** modeling for variable-length inputs and outputs

Sequence-to-Sequence Modeling

Sequence-to-sequence (Seq2Seq) model is fundamentally built on **two components**:

Encoder: An RNN (often LSTM or GRU) reads the input sequence and produces hidden states.

Decoder: another RNN (again LSTM or GRU) uses the encoder's final state to generate the output sequence step-by-step.

Seq2Seq forms the basis for advanced sequence-to-sequence models like Attention models, GPT Models, Transformers, and BERT.

Modern Seq2Seq

Later models introduced improvements:

- **BiLSTMs** in the encoder to capture both left and right context
- Attention mechanism (Bahdanau et al., 2015) to overcome **long-range dependency** issues
- Transformer-based **Seq2Seq** (Vaswani et al., 2017) without any RNNs

Road to Transformers

1. RNN Problems Recap
2. LSTM — Long Short-Term Memory
3. GRU — Gated Recurrent Units
4. Seq2Seq Models
5. Attention Mechanism
6. Self-Attention & Multi-Head
7. Transformers

Machine Translation

Machine Translation (MT) is the task of translating a sentence *x* from one language (the source language) to a sentence *y* in another language (the target language).

x: *L'homme est né libre, et partout il est dans les fers*

↓

y: *Man is born free, but everywhere he is in chains*

Machine Translation: 1950s

- Machine translation research began in the **early 1950s** on machines less powerful than high school calculators (before the term “A.I.” was coined!).
- MT heavily funded by military, but basically just simple rule-based systems doing word substitution.
- Human language is more complicated than that, and varies more across languages!
- Little understanding of natural language syntax, semantics, pragmatics.

Machine Translation: 1950s

- Rule-Based MT (RBMT)
- Statistical MT (SMT)
- Phrase-Based SMT

Limitations: rigid rules, feature engineering, weak generalization

2014 Neural Machine Translation

The 2014 work on **Neural Machine Translation (NMT)** marks a pivotal shift in machine translation (MT), introducing end-to-end neural architectures that replaced traditional statistical MT systems.

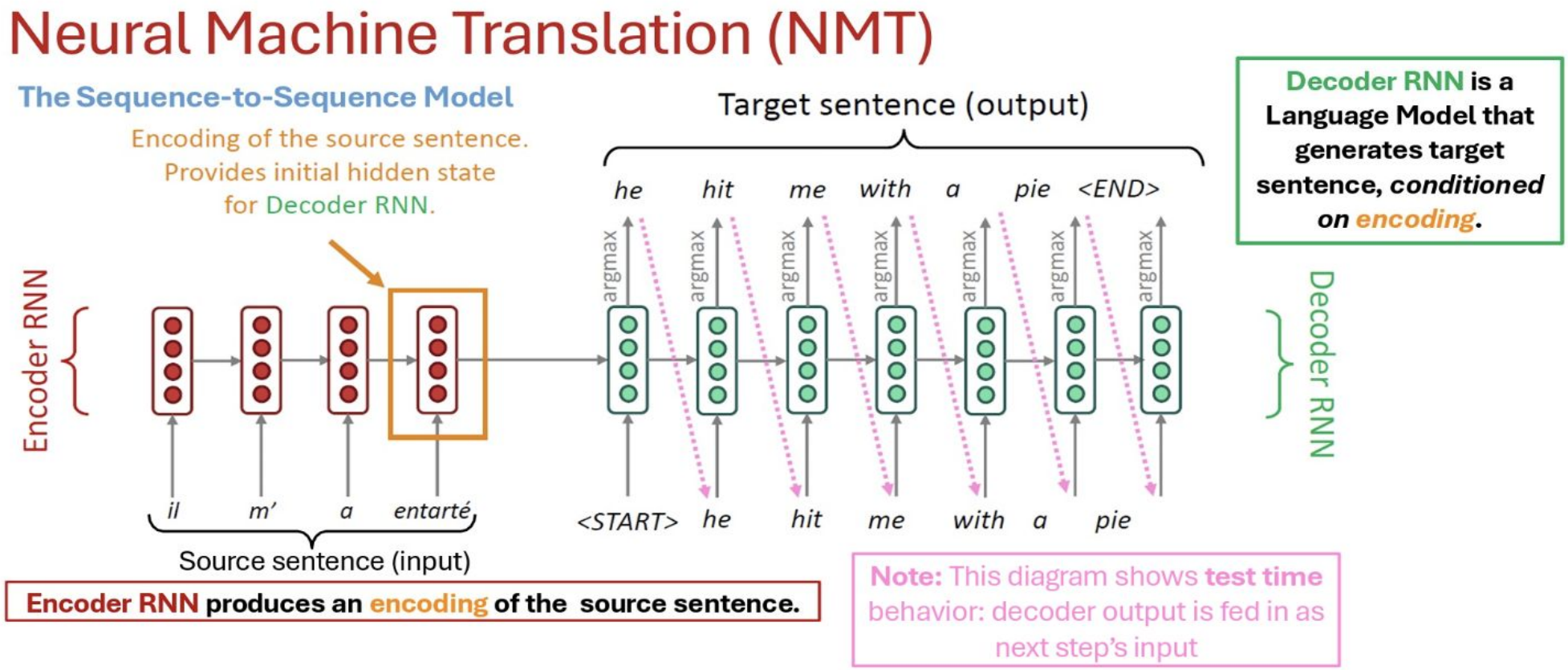
Two landmark papers introduced foundational ideas:

Sequence-to-Sequence Learning with Neural Networks (Sutskever et al., 2014, NIPS): *Demonstrated that a single neural network could be trained end-to-end for translation.*

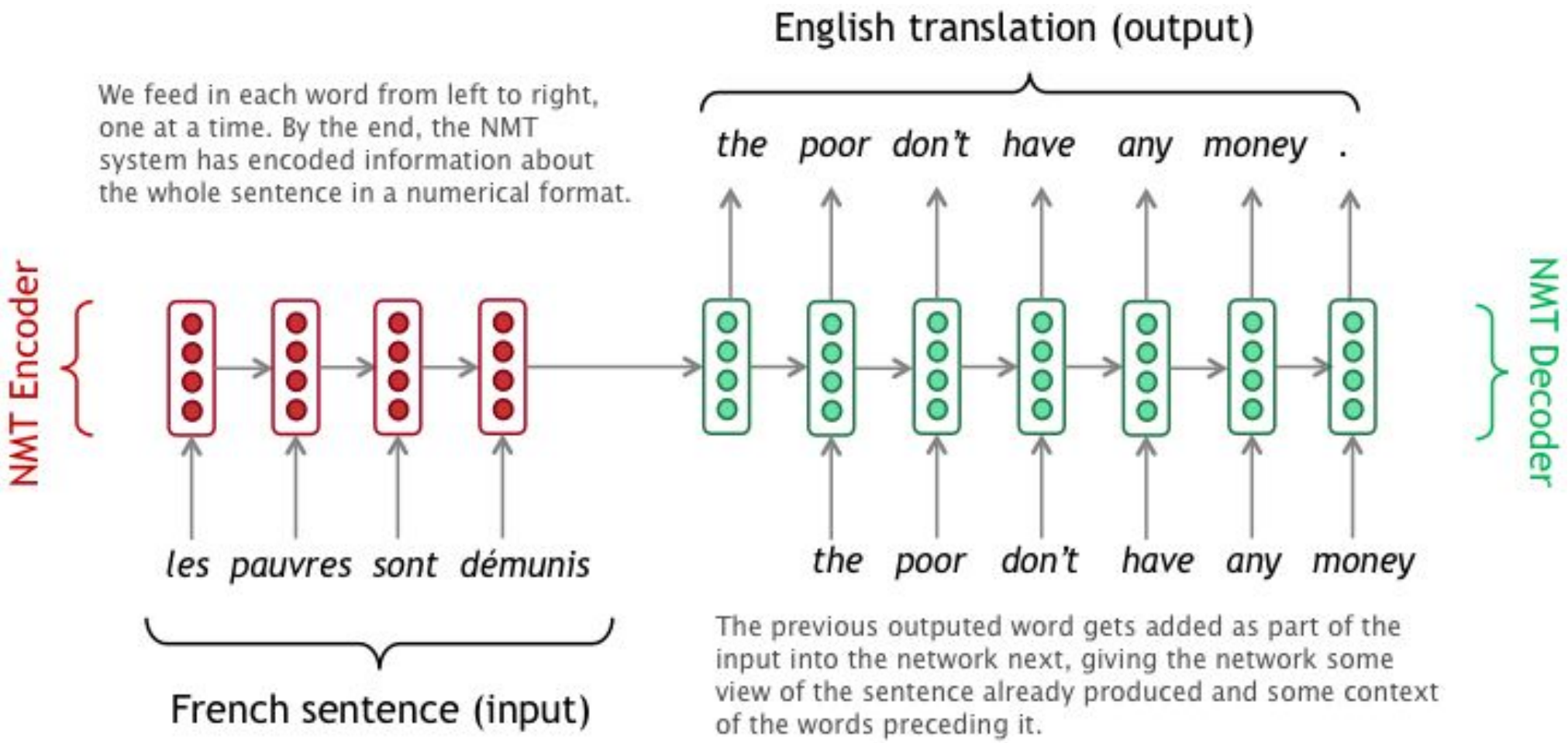
Neural Machine Translation by Jointly Learning to Align and Translate (Bahdanau et al., 2014, ICLR 2015) *Introduced the attention mechanism, allowing the decoder to dynamically focus on different parts of the input sequence at each output step.*

Neural Machine Translation

Neural Machine Translation



Neural Machine Translation



If the input and output are sequences, we call it seq2seq model (e.g., MT)

Neural Machine Translation

- The sequence-to-sequence model is an example of a **Conditional Language Model**
  - **Language Model** because the decoder is predicting the next word of the target sentence *y*
  - **Conditional** because its predictions are *also* conditioned on the source sentence *x*

- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y_1|x)\,P(y_2|y_1,x)\,P(y_3|y_1,y_2,x)\ldots\underbrace{P(y_T|y_1,\ldots,y_{T-1},x)}_{\text{Probability of next target word, given target words so far and source sentence } x}$$

- **Question:** How to train an NMT system?
- **(Easy) Answer:** Get a big parallel corpus…
  - But there is now exciting work on "unsupervised NMT", data augmentation, etc.

Decoding

After training a sequence-to-sequence model (e.g., for machine translation), **decoding** refers to how we **generate the output sequence** (e.g., translated sentence) from the model.

At each step, the decoder:

- Takes in previous generated words
- Outputs a probability distribution over the vocabulary
- Selects the next word

The choice of decoding strategy significantly affects translation quality.

Decoding Strategies

- Greedy Decoding
- Beam Search Decoding
- Sampling-Based Decoding

NMT: the first big success story of NLP Deep Learning

Neural Machine Translation went from a fringe research attempt in **2014** to the leading standard method in **2016**

- **2014**: First seq2seq paper published [Sutskever et al. 2014]
- **2016**: Google Translate switches from SMT to NMT – and by 2018 everyone had
  - https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html

Microsoft SYSTRAN beyond language Google facebook

Baidu 百度 網易 NetEase www.163.com Tencent 腾讯 搜狗搜索

- This was amazing!
  - **SMT** systems, built by hundreds of engineers over many years, were outperformed by NMT systems trained by small groups of engineers in a few months

African Languages lacks parallel corpora for MT.

Example of Parallel Dataset

English Sentence	French Translation
Hello, how are you?	Bonjour, comment allez-vous ?
I am learning machine learning.	J'apprends l'apprentissage automatique.
The cat sat on the mat.	Le chat était assis sur le tapis.
Where is the nearest train station?	Où se trouve la gare la plus proche ?
Thank you for your help.	Merci pour votre aide.
What time is it?	Quelle heure est-il ?
I love to read books.	J'adore lire des livres.
She is a talented artist.	C'est une artiste talentueuse.
We are going to the park.	Nous allons au parc.
Have a great day!	Passez une excellente journée !

What is parallel Corpora?

A parallel corpus is a collection of texts in ***two or more languages where each text is aligned with its translation(s).***

- Can be bilingual (two languages) or multilingual (more than two).

- Alignment can be:

  - **Unidirectional**: One language translated into another.
  - **Bidirectional**: Translations exist both ways.
  - **Multidirectional**: Text translated into multiple languages (e.g., EU regulations).

Requirements for a Good Parallel Corpus

A high-quality corpus should:

1. Contain authentic language data (naturally occurring, not artificially generated).
2. Be representative of different discourse types (formal, informal, technical, conversational, etc.).

Always check corpus quality before using it for training models.

Poor quality data → poor model performance.

Europarl: A Parallel Corpus for Statistical Machine Translation

Philipp Koehn
School of Informatics
University of Edinburgh, Scotland
pkoehn@inf.ed.ac.uk

Abstract

We collected a corpus of parallel text in 11 languages from the proceedings of the European Parliament, which are published on the web[1]. This corpus has found widespread use in the NLP community. Here, we focus on its acquisition and its application as training data for statistical machine translation (SMT). We trained SMT systems for 110 language pairs, which reveal interesting clues into the challenges ahead.

1 Introduction

We collected the Europarl corpus mainly to aid our research in statistical machine translation, but since we made it available in its initial release in 2001, it has been used for many other natural language problems: word sense disambiguation, anaphora resolution, information extraction, etc.

This paper describes the acquisition of the corpus and its application to the task of statistical machine translation. We used the corpus to build 110 machine translation systems for all the possible language pairs. The resulting systems and their performances demonstrate the different challenges for

A Few Thousand Translations Go A Long Way! Leveraging Pre-trained Models for African News Translation

David Ifeoluwa Adelani[1*], Jesujoba Oluwadara Alabi[2*], Angela Fan[3*], Julia Kreutzer[4*], Xiaoyu Shen[5], Machel Reid[6*], Dana Ruiter[1], Dietrich Klakow[1], Peter Nabende[7*], Ernie Chang[1*], Tajuddeen R. Gwadabe[8*], Freshia Sackey[9*], Bonaventure F. P. Dossou[10*], Chris Chinenye Emezue[11*], Colin Leong[12*], Michael Beukman[13*], Shamsuddeen H. Muhammad[14*], Guyo D. Jarso[*], Oreen Yousuf[15*], Andre N. Rubungo[16*], Gilles Hacheme[17*], Eric P. Wairagala[7*], Muhammad U. Nasir[18*], Benjamin A. Ajibade[*], Tunde Oluwaseyi Ajayi[*], Yvonne Wambui Gitau[*], Jade Abbott[*], Mohamed Ahmed[19*], Millicent Ochieng[19*], Anuoluwapo Aremu[*], Perez Ogayo[20*], Jonathan Mukiibi[7*], Fatoumata Ouoba Kabore[*], Godson Koffi Kalipe[*], Derguene Mbaye[21*] Allahsera Auguste Tapo[22*], Victoire M. Koagne[*], Edwin Munkoh-Buabeng[*], Valencia Wagner[23*], Idris Abdulmumin[24*], Ayodele Awokoya[25*], Happy Buzaaba[*], Blessing Sibanda[26*], Andiswa Bukula[27*], Sam Manthalu[28]

[*]Masakhane NLP, [1]Saarland University, Germany, [2]Inria, France, [3]Meta AI, [4]Google Research, [5]Amazon Alexa AI, [6]The University of Tokyo, Japan, [7]Makerere University, Kampala, Uganda, [8]UCAS, China, [9] JKUAT, Kenya, [10]Jacobs University, Germany, [11]TUM, Germany, [12]University of Dayton, USA, [13]University of the Witwatersrand, South Africa, [14]LIAAD-INESC TEC, Porto, Portugal, [15]Uppsala University, Sweden, [16]UPC, Spain, [17]Ai4Innov [18]Ominor AI [19]Microsoft Africa Research Institute, Kenya [20]CMU, USA, [21]Baamtu, [22]RIT, USA, [23]SPU, South Africa, [24]ABU, Nigeria, [25]UI Ibadan, Nigeria, [26]NUST, Namibia [27]SADiLaR, South Africa, [28]University of Malawi, Malawi

Abstract

Recent advances in the pre-training of language models leverage large-scale datasets to create multilingual models. However, low-resource languages are mostly left out in these datasets. This is primarily because many widely spoken languages are not well represented on the web and therefore excluded from the large-scale crawls used to create datasets. Furthermore, downstream users of these models are restricted to the selection of languages originally chosen for pre-training. This work investigates how to optimally leverage existing pre-trained models to create low-resource translation systems for 16 African languages. We focus on two questions: 1) *How can pre-trained models be used for languages not included in the initial pre-training?* and 2) *How can the resulting translation models effectively transfer to new domains?* To answer these questions, we create a new African news corpus cover-

while leveraging the maximal amount of data for training, most prominently large crawls of monolingual and parallel data from the web (El-Kishky et al., 2020; Schwenk et al., 2021b,a; Xue et al., 2021b). The resulting models are now capable of translating between hundreds of languages, including language pairs that in isolation do not have large collections of parallel data (Tang et al., 2020; Xue et al., 2021a; Fan et al., 2021b). For example, M2M-100 (Goyal et al., 2021) can translate (with low accuracy) between Hausa and Yorùbá, two of the most widely spoken languages in Nigeria, even though there is barely any parallel data available for training. For languages that are not included in the set of training languages, the model would have no knowledge on how to generate translations. Does this mean there is no hope for languages that do not have large presence on the web and are therefore not included in these pre-trained models?

https://github.com/masakhane-io/lafand-mt

Sometimes data is not available, people explore other approaches such as Data Augumentations

Data Augmentation

Data augmentation is the process of artificially expanding a dataset by creating new examples from existing ones.

- Helps improve model generalization
- Reduces overfitting in low-resource tasks
- *Particularly critical in low-resource NLP, where labeled data is scarce*

Useful in many NLP task such as Machine Translation (MT), Sentiment Analysis (SA), Hate Speech Detection, Named Entity Recognition (NER)

Goals of Data Augmentation

1. Increase dataset size: more examples to learn from
2. Improve model robustness: model becomes less sensitive to noise
3. Introduce linguistic variety: helps models generalize to unseen forms
4. Address class imbalance: oversample minority classes

Data Augmentation in Machine Translation

Challenge: MT needs parallel corpora; generating new parallel sentences is difficult.

Common strategies:

- Back-Translation
  - Translate target language sentence back to source → creates synthetic source sentence
  - Example: French → English → French
- Paraphrasing
  - Rephrase source or target sentence using NLP models
  - Maintains meaning but changes wording

Data Augmentation in Machine Translation

Noising / Token-level perturbation

- Swap, delete, or insert words in the source sentence
- Helps model learn robustness to typos or variation

Round-trip translation

- Source → Pivot language → Source
- Generates diverse source examples for training

Example of Back-Translation

The cat is sleeping Le chat dort

Back-translation steps:

1. Train a reverse MT model (French → English)
2. Translate French back to English → “The kitty sleeps”
3. Pair “The kitty sleeps” with original French → synthetic training pair

Benefit: Increases dataset size with realistic variation

Tools and Libraries for NLP Augmentation

- NLPAug (Python): supports word, character, and contextual augmentation
- TextAttack: adversarial and augmentation strategies for text
- OpenNMT / MarianMT: back-translation for MT
- Hugging Face Transformers: paraphrasing and synthetic generation

Data Augmentation in Machine Translation

Enhanced back-translation for low resource neural machine translation using self-training*

Idris Abdulmumin[1,2][0000−0002−3795−8381], Bashir Shehu Galadanci[2], and Abubakar Isa[1]

[1] Ahmadu Bello University, Samaru Campus, Zaria, Kaduna State, Nigeria
{iabdulmumin, abubakarisa}@abu.edu.ng
[2] Bayero University, Kano, Kano State Nigeria
bsgaladanci.se@buk.edu.ng

Abstract. Improving neural machine translation (NMT) models using the back-translations of the monolingual target data (synthetic parallel data) is currently the state-of-the-art approach for training improved translation systems. The quality of the backward system – which is trained on the available parallel data and used for the back-translation – has been shown in many studies to affect the performance of the final NMT model. In low resource conditions, the available parallel data is usually not enough to train a backward model that can produce the qualitative synthetic data needed to train a standard translation model. This work proposes a self-training strategy where the output of the backward model is used to improve the model itself through the forward translation technique. The technique was shown to improve baseline low resource IWSLT'14 English-German and IWSLT'15 English-Vietnamese backward translation models by 11.06 and 1.5 BLEUs respectively. The synthetic data generated by the improved English-German backward model was used to train a forward model which out-performed another forward model trained using standard back-translation by 2.7 BLEU.

Synthetic Data Generation with LLM

Use pretrained models (e.g., GPT, LLaMA) to generate synthetic examples.

Example: Prompting with task instructions:

"Generate 5 positive Hausa movie reviews"

Pros: High diversity and natural language output
Cons: Risk of noise or bias in generated data

Synthetic Data Generation with LLM

Scaling Low-Resource MT via Synthetic Data Generation with LLMs

Ona de Gibert **Joseph Attieh** **Teemu Vahtola** **Mikko Aulamo** **Zihao Li**
Raúl Vázquez **Tiancheng Hu** **Jörg Tiedemann**
University of Helsinki University of Cambridge
{first.last}@helsinki.fi, th656@cam.ac.uk

Abstract

We investigate the potential of LLM-generated synthetic data for improving low-resource Machine Translation (MT). Focusing on seven diverse target languages, we construct a document-level synthetic corpus from English Europarl, and extend it via pivoting to 147 additional language pairs. Automatic and human evaluation confirm its overall high quality. We study its practical application by (i) identifying effective training regimes, (ii) comparing our data with the HPLT dataset, (iii) studying the effect of varying training data size, and (iiii) testing its utility beyond English-centric MT. Finally, we introduce SynOPUS, a public repository for synthetic parallel datasets. Our findings show that LLM-generated synthetic data, even when noisy, can substantially improve MT performance for low-resource languages.

no systematic investigation of (a) generating large-scale synthetic data using LLMs for low-resource languages, (b) evaluating its intrinsic quality, (c) quantifying its downstream impact when training or fine-tuning modern MT systems. This paper provides such a systematic investigation. We make the following contributions:

- We use GPT-4o[1] to generate a document-level synthetic parallel corpus by forward-translating English Europarl (Koehn, 2005) into seven diverse low-resource languages.
- We assess the corpus quality using both automatic metrics and human evaluation, finding the data to be generally of high quality.
- We comprehensively evaluate the utility of this synthetic data by demonstrating that:

  (1) Compact MT models trained from scratch

Synthetic Voice Data for Automatic Speech Recognition in African Languages

Brian DeRenzi
Dimagi
bderenzi@dimagi.com

Anna Dixon
Dimagi*
anna@datafoss.com

Mohamed Aymane Farhi
CLEAR Global
aymenfarhi.25@gmail.com

Christian Resch†
CLEAR Global
christian.resch@clearglobal.org

Abstract

Speech technology remains out of reach for most of the 2,300+ languages in Africa. We present the first systematic assessment of large-scale synthetic voice corpora for African ASR. We apply a three-step process: LLM-driven text creation, TTS voice synthesis, and ASR fine-tuning. Eight out of ten languages for which we create synthetic text achieved readability scores above 5 out of 7. We evaluated ASR improvement for three (Hausa, Dholuo, Chichewa) and created more than 2,500 hours of synthetic voice data at below 1% of the cost of real data. W2v-BERT 2.0 speech encoder fine-tuned on 250h real and 250h synthetic data in Hausa matched a 500h real-data-only baseline, while 579h real and 450h to 993h synthetic data created the best performance. We also present gender-disaggregated ASR performance evaluation. For very low-resource languages, gains varied: Chichewa WER improved by ∼6.5% with a 1:2 real-to-synthetic ratio; a 1:1 ratio for Dholuo showed similar improvements on some evaluation data, but not on others. Investigating intercoder reliability, ASR errors and evaluation datasets revealed the need for more robust reviewer protocols and more accurate evaluation data. All data and models are publicly released to invite further work to improve synthetic data for African languages.

supported languages, incurring substantial costs estimated at more than US$100–150 per hour, even in the best case[1]. As most speech recognition models need several hundred hours of training data to achieve performance sufficient for practical application, with current investments in AI for development, human data collection is prohibitively costly to cover the many languages that remain unsupported. We support further investment in African language technology, but even if it were to become available, those investments have opportunity costs, diverting funds that could otherwise be spent on other interventions.

Therefore, we are investigating synthetic voice data as a complementary approach to create and improve automatic speech recognition (ASR) in African languages. The principle hypothesis motivating our work is that we can leverage Large Language Models (LLMs) and Text-to-Speech (TTS) models to create synthetic voice data of sufficient quality to improve automatic speech recognition models. Our work shows that this synthetic voice data can be created for less than 1% of the cost of collecting real human data[2], while holding potential to complement this human data in creating and improving ASR models for African languages.

2 Synthetic Data: Risks and Rewards

MT Evaluation is difficult

- A sentence may have multiple correct translations
- Languages differ in syntax and word order
- Literal matching may not reflect semantic correctness

Human Evaluation

- The most reliable evaluation approach.
- Human annotators rate translations based on:
  - **Adequacy:** How much meaning from the source is preserved.
  - **Fluency:** How natural the translation sounds in the target language.

Example scoring scale:

5 - Perfect translation

3 - Understandable but imperfect

1 - Unusable translation

Human Evaluation

Advantages

- Most accurate

Limitations

- Expensive
- Slow
- Not scalable

Therefore, automatic metrics are widely used.

Automatic Evaluation

Automatic evaluation metrics fall into three major groups:

1. Lexical Overlap Metrics
2. Edit Distance Metrics
3. Semantic Metrics

Lexical Overlap Metrics

Compare word overlap between system output and reference translation.

Examples:

- BLEU
- ROUGE

BLEU (Bilingual Evaluation Understudy)

One of the most widely used MT metrics.

Core Idea: Measure n-gram overlap between:

- system translation
- reference translation

BLEU computes precision of:

- unigram
- bigram
- trigram
- four-gram matches

Example

Reference: The cat is on the mat
System output: The cat sits on the mat

Matching n-grams increase the BLEU score.

Edit Distance Metric

Measure the number of edits required to transform system output into reference.

Example: TER

Number of edits required to transform system output into reference translation Possible edits: Insertions Deletions Substitutions Word shifts

Example

Reference: The cat is on the mat
System output: Cat is on the mat

Edit needed:
Insert “The”

TER = Number of edits / sentence length

Lower TER is better.

Semantic Metrics

Evaluate semantic similarity using embeddings or neural models.

Examples:

- BERTScore
- COMET

BERTScore

A modern semantic evaluation metric. Instead of word overlap,

BERTScore compares:

- contextual embeddings from transformer models

Steps:

1. Encode system translation and reference using BERT
2. Compute similarity between word embeddings
3. Aggregate similarity scores

Advantages:

- Captures semantic similarity
- Handles synonyms and paraphrases

COMET

A neural evaluation metric trained to predict human judgments.

COMET uses:

- Source sentence
- System translation
- Reference translation

Input into a neural model → predicted quality score.

Advantages:

- High correlation with human evaluation
- Strong performance across languages

COMET

The source sentence is the original sentence written in the source language that needs to be translated by the machine translation system.

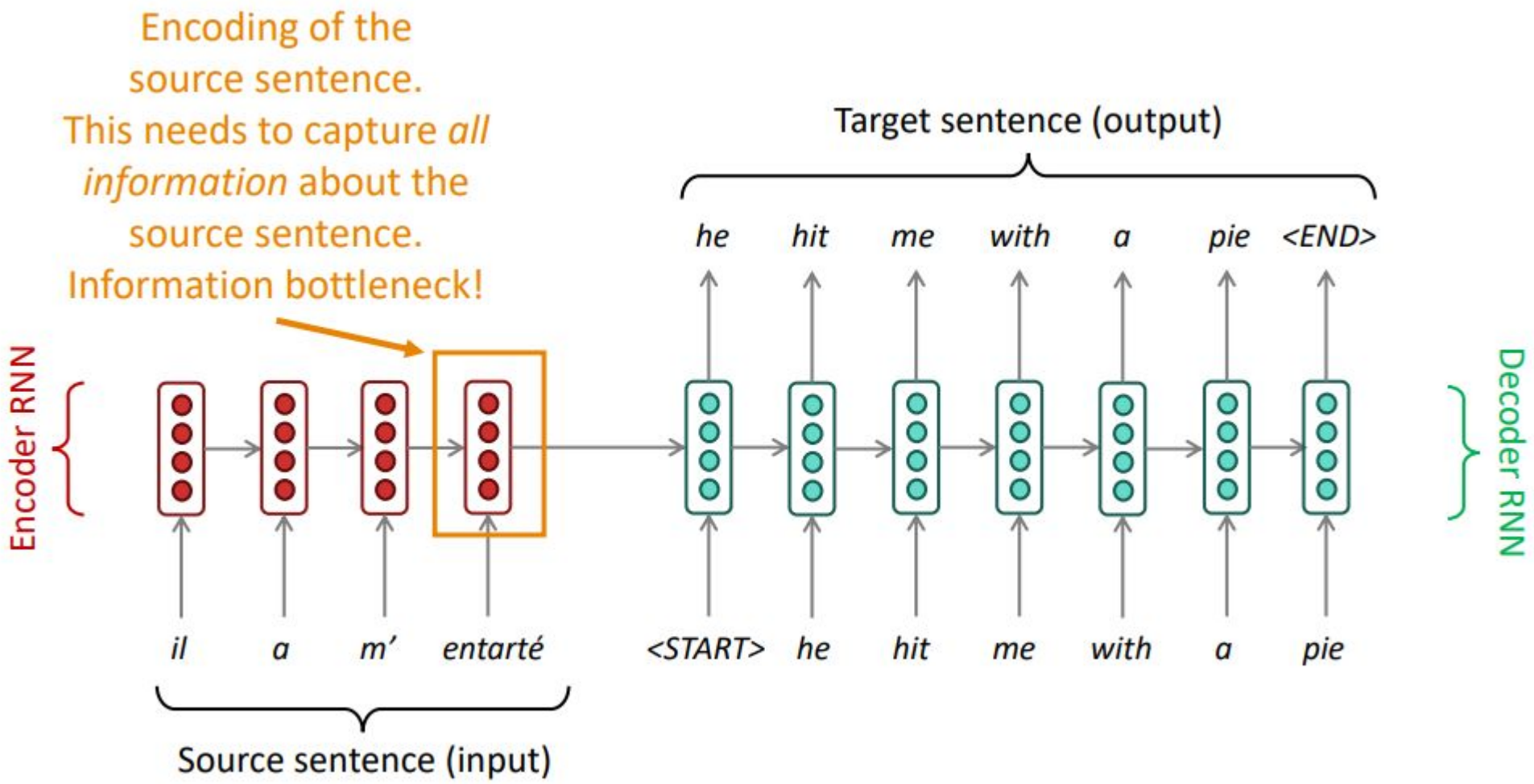
The system translation—often called the hypothesis—is the sentence produced by the machine translation model after processing the source sentence.

The reference translation is the correct translation written by a human translator.

Road to Transformers

1. RNN Problems Recap
2. LSTM — Long Short-Term Memory
3. GRU — Gated Recurrent Units
4. Seq2Seq Models
5. Attention Mechanism
6. Self-Attention & Multi-Head
7. Transformers

Bottleneck Problem in Encoder-Decorder



Bottleneck Problem in Encoder-Decorder

- The entire source sentence must be compressed into **ONE fixed-size vector**
- For short sentences: **OK** — the vector is sufficient
- For long sentences (**50, 100, 200+ words):** IMPOSSIBLE to preserve all information
- Bahdanau et al. (2015) showed: translation quality drops sharply as sentence length grows
- The bottleneck problem: '***How much can you compress a novel into a single page?'***

BLEU Score vs. Sentence Length:

Short (1-10 words): High quality | Medium (20-30): Decent | Long (50+): Quality COLLAPSES

Bottleneck Problem in Encoder-Decorder

Bottleneck Problem Issue: In encoder-decoder RNNs (e.g., Seq2Seq), the entire input sequence is compressed into a fixed-size **context vector**.

Consequence: This "bottleneck" limits the amount of information passed from encoder to decoder, especially for long input sequences, leading to poor performance in tasks like translation or summarization.

Attention Mechanism

IMPERIAL

Q and A